Problem 10: Caching (5 + 20 = 25 points) [File Upload]

By Jishnu Roychoudhury

> *My biggest mistake was using the C syntax. A lot of people are very proud of writing functions which return function pointers... you shouldn't be proud of such things.*
> *– Bjarne Stroustrup, creator of C++, one week ago.*

Problem Statement

You're asked to implement a key-value cache in C. Your cache will be evaluated based on the number of cache misses.

Background: Caching

When you load a website, your web browser saves copies of images, scripts, and other data so that the page loads faster the next time. This saved data is stored in a cache. If the browser needs something and it is already in the cache, that is a **cache hit**. If it is not there and must be fetched again, that is a **cache miss**.

Caching is a general technique used to speed up data access by keeping a small amount of frequently used information in a convenient place. Because caches have limited capacity, they must decide which item to remove when they are full. This is handled by a **cache-replacement policy**, such as FIFO or LRU.

A **key–value cache** is a cache where each stored item is identified by a unique **key** and is associated with some **value**. When a key is requested, the cache checks whether it is already stored:

- If the key is present, the value is returned immediately (a hit).
- If the key is not present, the cache reports a miss and must load the key–value pair from elsewhere.

In this problem, you will implement such a key–value cache.

CITADEL | CITADEL Securities
DE Shaw & Co
Jane Street®
PRINCETON Computer Science
SCM STEVENS CAPITAL MANAGEMENT
ORFE
Department of Mathematics

Problem Details

You are asked to implement three functions: `cache_init`, `cache_get`, and `cache_store`.

In this problem, we emulate the interaction between "cache" and "memory" by having your program (the cache) interact with a grader program (the memory). Specifically, your cache program will receive a sequence of 10^6 keys. First, the grader calls `cache_init`, allowing you to set up your cache. Then, for each key, the following sequence of actions will occur:

- The grader calls `cache_get` with parameter `key`.
- In `cache_get`, you should respond with the value corresponding to `key` (if you know it), or 0 (indicating that you do not know it). Note that if you return a nonzero incorrect value your program will be judged incorrect.
- In the first case, this is a cache hit, and the grader will move onto the next key. In the second case, the grader will call `cache_store`, and you can store the key.

Each key is a string of length at most 64 characters. Each value is a 16-bit signed integer. All keys are composed of only lowercase letters.

Since you are implementing a cache, your program will be restricted in memory. We allow your program to request at most 16KiB of heap memory. That is, at any one time, your program should hold at most 16KiB of heap memory. To enforce this, we request that you use `cmalloc` for allocation and `cfree` for deallocation. Any program which uses standard `malloc`, `free`, `realloc`, or any other workaround, will score 0. In addition, we allow your program to use at most 1KiB of stack memory. In particular, large local arrays, deep recursion, or large stack-allocated structs are not allowed and will cause your submission to score 0.

We run your program in C17. Download the files to get the environment and starter code. Do not make any changes to the starter code except or where indicated. You can compile your code with the following command:

```
clang -std=c17 -Wall -Wextra grader.c contestant.c alloc.c -o contestant
```

You can run your code on an input with the following command: `./contestant < input.txt`

CITADEL | CITADEL Securities
DE Shaw & Co
Jane Street®
PRINCETON Computer Science
SCM STEVENS CAPITAL MANAGEMENT
ORFE
Department of Mathematics

Scoring

We will evaluate your solution on two testcases. For each testcase, if you ever respond with a nonzero incorrect value (or break the rules listed above), you get 0 points.

The first testcase is worth 5 points. For this testcase, keys have length at most 2 characters. Your score for this testcase is $\min\{5, 2 + \log_{10}(\frac{10^6}{N})\}$, where N is the number of cache misses. In particular, you get 2 points for any correct code (even if it misses every time), and a maximum of 3 more points if your code makes 1000 or less cache misses.

The second testcase is worth 20 points. The testcase emulates a realistic sequence of accesses for a real key-value cache. Your score for this testcase is $20\left(\frac{10^6-N}{10^6}\right)$, where N is the number of cache misses. We have a solution which makes around 215,000 cache misses for a score of 15.7.

We provide input files which have the same properties as the two testcases we use (but are different; do not try to reverse-engineer the testcases). You can use these to evaluate your performance locally. You can submit to us at most 5 times.

CITADEL | CITADEL Securities
D E Shaw & Co
Jane Street®
PRINCETON Computer Science
SCM STEVENS CAPITAL MANAGEMENT
ORFE
Department of Mathematics